Introduction à JavaScript

- Introduction à JavaScript.
- Types de données et opérateurs.
- Structures de contrôle.
- Les Fonctions.
- Les Tableaux.
- DOM et manipulation du DOM.
- Les Événements.

- JavaScript a été créé en 1995 par Netscape Communications pour ajouter de la interactivité aux sites web.
- Le langage a été rendu public et a été standardisé par l'ECMA International sous le nom de ECMAScript.
- Au fil des années, de nouvelles versions de JavaScript ont été publiées, avec de nouvelles fonctionnalités et des améliorations de performance.

- JavaScript s'est rapidement popularisé et est devenu le principal langage de programmation côté client pour le développement web.
- Il est maintenant largement utilisé pour créer des applications web interactives et riches en fonctionnalités, ainsi que pour développer des applications mobiles et de bureau.
- JavaScript est également utilisé dans de nombreuses autres domaines en dehors du développement web, comme la robotique, l'Internet des objets (IoT) et l'analyse de données.

- Un moteur JavaScript est un logiciel qui exécute du code JavaScript.
- Il existe plusieurs moteurs JavaScript différents, chacun conçu pour être utilisé dans un environnement spécifique, comme un navigateur web, une application mobile ou un serveur.
- Les moteurs JavaScript sont responsables de la compilation et de l'exécution du code JavaScript.

- Les moteurs JavaScript analysent le code et le traduisent en instructions que le processeur de l'ordinateur peut exécuter.
- Les moteurs JavaScript modernes sont conçus pour être efficaces et rapides, avec des optimisations pour la compilation et l'exécution du code.
- En tant que développeur, vous n'avez généralement pas besoin de vous soucier du moteur JavaScript utilisé, car votre code s'exécutera de la même manière quelle que soit la plateforme sur laquelle il est exécuté.

- Voici quelques-uns des moteurs JavaScript les plus populaires:
  - **V8**: Ce moteur JavaScript open source est utilisé par Google Chrome et Node.js. Il est connu pour être rapide et efficace, grâce à des optimisations de compilation et d'exécution de code avancées.
  - **Spider Monkey**: Ce moteur JavaScript est utilisé par Mozilla Firefox. Il a été l'un des premiers moteurs JavaScript open source et a été largement utilisé dans les premières versions de Firefox.
  - **Chakra**: Ce moteur JavaScript est utilisé par Microsoft Edge et Internet Explorer. Il est connu pour sa compatibilité avec les normes web et son support pour les fonctionnalités avancées de JavaScript, comme les classes et les générateurs.
  - **JavaScriptCore**: Ce moteur JavaScript est utilisé par Safari et WebKit. Il est connu pour son support de la norme ECMAScript et pour sa performance sur les appareils mobiles.

- Comment inclure JavaScript dans une page web ?
  - Il existe plusieurs façons d'inclure le code JavaScript dans une page web.
  - La méthode la plus courante consiste à utiliser une balise *<script>* dans le code HTML de la page.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>EST SALE</title>
  </head>
  <body>
    <script>
      alert("Bonjour GI 1 !!");
    </script>
  </body>
</html>
```

- En utilisant l'inclusion de fichiers externes: on peut inclure du code JavaScript dans une page web en faisant référence à un fichier externe contenant le code.
- Pour effectuer ça on utilise la balise *script* et l'attribut *src*. Voici un exemple de code qui inclut un fichier externe de JavaScript dans une page web:

```html
<!DOCTYPE html>
<html>
  <head>
    <title>EST SALE</title>
  </head>
  <body>
    <script src="./script.js"></script>
  </body>
</html>
```

- **Les types de données:**

  - Nombres: Les nombres peuvent être entiers ou décimaux et sont utilisés pour représenter des valeurs numériques.
  - Chaînes de caractères: Les chaînes de caractères sont utilisées pour représenter du texte et sont entourées de guillemets simples ou doubles.
  - Booléens: Les booléens sont utilisés pour représenter les valeurs vrai ou faux.
  - Null: La valeur null représente l'absence de valeur.
  - Undefined: La valeur undefined représente une variable qui a été déclarée, mais qui n'a pas encore été assignée à une valeur.

JS

- **Les opérateurs arithmétiques:**
  - Addition (+)
  - Soustraction (-)
  - Multiplication (*)
  - Division (/)
  - Modulo (%)
- **Les opérateurs logique:**
  - ET (&&)
  - OU (| |)
  - NON (!)

JS

- **Les opérateurs de comparaison:**
  - Égal à (==)
  - Strictement égal à (===)
  - Différent de (!=)
  - Strictement différent de (!==)
  - Plus grand que (>)
  - Plus grand ou égal à (>=)
  - Plus petit que (<)
  - Plus petit ou égal à (<=)
- **Autres opérateurs**
  - Affectation (=)
  - Opérateurs d'incrémentation/décrémentation (++/--)
  - Opérateur ternaire (condition ? valeur si vrai : valeur si faux)

- **L'instruction if:**
- L'instruction if permet de tester une condition et d'exécuter un bloc de code si la condition est vraie.
- Syntaxe:

```
if (condition) {
  // code à exécuter si la condition est vraie
}
```

- Exemple:

```
let temp = 30;
if (temp > 32) {
  console.log("Il fait chaud !");
}
```

- **L'instruction else:**
- L'instruction else permet d'exécuter un bloc de code si la condition testée par l'instruction if est fausse.
- Syntaxe:

```
if (condition) {
  // code à exécuter si la condition est vraie
} else {
  // code à exécuter si la condition est fausse
}
```

- Exemple:

```
let temp = 30;
if (temp > 32) {
  console.log("Il fait chaud !");
} else {
  console.log("Il fait froid !");
}
```

- **L'instruction else if:**
- L'instruction else if permet de tester une nouvelle condition si la condition testée par l'instruction if est fausse. Elle peut être utilisée pour tester plusieurs conditions de manière successive.
- Syntaxe:

```
if (condition1) {
  // code à exécuter si la condition1 est vraie
} else if (condition2) {
  // code à exécuter si la condition1 est fausse et que la condition2 est vraie
} else {
  // code à exécuter si aucune des conditions n'est vraie
}
```

- Exemple:

```
let temp = 30;
if (temp > 32) {
  console.log("Il fait chaud !");
} else if (temp < 0) {
  console.log("Il fait très froid !");
} else {
  console.log("Il fait frais !");
}
```

- **L'instruction switch:**
- L'instruction switch permet de tester une valeur et d'exécuter un bloc de code en fonction de la valeur testée. Elle peut être utilisée lorsqu'il y a plusieurs valeurs possibles pour une variable et que vous souhaitez exécuter un bloc de code différent en fonction de chaque valeur.
- Syntaxe:

```
switch (expression) {
  case valeur1:
    // code à exécuter si expression vaut valeur1
    break;
  case valeur2:
    // code à exécuter si expression vaut valeur2
    break;
  default:
    // code à exécuter si aucune des valeurs n'est trouvée
}
```

- **L'instruction switch:**
- Exemple:

```
let fruit = "pomme";

switch (fruit) {
  case "banane":
    console.log("Le fruit est une banane.");
    break;
  case "pomme":
    console.log("Le fruit est une pomme.");
    break;
  case "orange":
    console.log("Le fruit est une orange.");
    break;
  default:
    console.log("Je ne connais pas ce fruit.");
}
// Affiche "Le fruit est une pomme."
```

- Dans cet exemple, la valeur de la variable fruit est testée et, si elle correspond à l'une des valeurs prévues dans les différents case, un message est affiché en conséquence. Si aucune des valeurs ne correspond, le bloc de code du default est exécuté.

- **La Structure while:**
- La boucle while est une structure de contrôle en JavaScript qui permet de répéter un groupe d'instructions tant qu'une condition est vraie.
- Voici un exemple simple de boucle while :

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

- Dans cet exemple, la boucle while exécute le code qui se trouve à l'intérieur de ses accolades tant que la valeur de i est inférieure à 5. À chaque itération de la boucle, la valeur de i est incrémentée de 1 grâce à l'instruction i++. Ainsi, la boucle affiche les valeurs de i de 0 à 4.
- Il est important de veiller à inclure une instruction qui modifie la condition de la boucle, afin d'éviter une boucle infinie. Dans l'exemple ci-dessus, cela se fait grâce à l'instruction i++.

- **La Structure do while:**
- La boucle do...while est une structure de contrôle similaire à la boucle while, mais elle a la particularité de s'exécuter au moins une fois, même si la condition est fausse.
- Voici un exemple simple de boucle do..while :

```
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 5);
```

- Dans cet exemple, la boucle do...while exécute le code qui se trouve à l'intérieur de ses accolades au moins une fois, puis vérifie la condition de la boucle. Si la condition est vraie, le code est exécuté à nouveau. Cela se poursuit tant que la condition est vraie. Dans cet exemple, la boucle affiche les valeurs de i de 0 à 4.

- **La Structure for:**
- La boucle for est une structure de contrôle en qui permet de répéter un groupe d'instructions pour un nombre défini de fois. Elle se compose de trois parties : l'initialisation, la condition et l'incrémentation.
- Voici un exemple simple de boucle for:

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

- Dans cet exemple, la boucle for commence par initialiser la variable i à 0. Ensuite, elle vérifie si la valeur de i est inférieure à 5. Si c'est le cas, le code qui se trouve à l'intérieur de ses accolades est exécuté. Après chaque itération, la valeur de i est incrémentée de 1 grâce à l'instruction i++. Cela se poursuit jusqu'à ce que la valeur de i soit supérieure ou égale à 5, ce qui met fin à la boucle. Dans cet exemple, la boucle affiche les valeurs de i de 0 à 4.

- **Les Fonctions:**
- Une fonction est un morceau de code qui peut être exécuté plusieurs fois à différents endroits d'un programme.
- Elle permet de regrouper des instructions qui accomplissent une tâche spécifique et de les réutiliser facilement.
- On peut également passer des valeurs à une fonction sous forme de paramètres et obtenir une valeur en retour grâce à l'instruction return.

```
function add(x, y) {
  return x + y;
}

const result = add(2, 3);
console.log(result); // affiche 5
```

- Dans cet exemple, la fonction add prend en entrée deux valeurs x et y et renvoie leur somme grâce à l'instruction return. On peut ensuite appeler la fonction en utilisant l'opérateur () et en lui passant des arguments, qui seront affectés aux paramètres de la fonction.

- **Les Tableaux :**
  - Un tableau en JavaScript est une collection d'éléments qui peuvent être de n'importe quel type de données (chaine de caractères, nombres, objets, etc.).
  - Les éléments d'un tableau sont stockés dans des cases qui ont un index numérique, commençant à 0 pour la première case.
  - Il y a plusieurs façons de créer un tableau en JavaScript :
    - En utilisant le constructeur **Array()** :

      let tab = new Array();

    - En utilisant une notation entre crochets **[]** :

      let tab = [];

    - En spécifiant les éléments du tableau entre crochets **[]**, séparés par une virgule :

      let tab = [1,2,3];

- **Les Tableaux :**
  - Pour accéder à un élément d'un tableau, on peut utiliser l'opérateur **[]** et indiquer l'index de l'élément à récupérer

```
let monTableau = [1, 2, 3, 4, 5];
let premierElement = monTableau[0]; // premierElement vaut 1
let dernierElement = monTableau[monTableau.length - 1]; // dernierElement vaut 5
```

  - Pour parcourir un tableau en JavaScript on peut utiliser une boucle **For**:

```
let monTableau = [1, 2, 3, 4, 5];
for (let i = 0; i < monTableau.length; i++) {
  console.log(monTableau[i]);
}
```

  - Voici un tableau récapitulatif des méthodes couramment utilisées pour manipuler les tableaux en JavaScript

Méthode	Description	Exemple
push	Ajoute un ou plusieurs éléments à la fin d'un tableau.	let arr = [1, 2, 3]; arr.push(4, 5, 6); console.log(arr); // [1, 2, 3, 4, 5, 6]
unshift	Ajoute un ou plusieurs éléments au début d'un tableau.	let arr = [1, 2, 3]; arr.unshift(4, 5, 6); console.log(arr); // [4, 5, 6, 1, 2, 3]
pop	Retire le dernier élément d'un tableau et le renvoie.	let arr = [1, 2, 3]; let last = arr.pop(); console.log(arr); // [1, 2] console.log(last); // 3
shift	Retire le premier élément d'un tableau et le renvoie.	let arr = [1, 2, 3]; let first = arr.shift(); console.log(arr); // [2, 3] console.log(first); // 1
slice	Renvoie un nouveau tableau qui contient une copie d'une portion du tableau original, allant de start jusqu'à end (exclu).	let arr = [1, 2, 3, 4, 5]; let slice = arr.slice(1, 3); console.log(arr); // [1, 2, 3, 4, 5] console.log(slice); // [2, 3]
concat	Renvoie un nouveau tableau qui contient la concaténation des tableaux passés en argument.	let arr1 = [1, 2, 3]; let arr2 = [4, 5, 6]; let arr3 = arr1.concat(arr2); console.log(arr1); // [1, 2, 3] console.log(arr3); // [1, 2, 3, 4, 5, 6]
sort	Trie les éléments d'un tableau selon la fonction de comparaison passée en argument.	let arr = [3, 2, 1]; arr.sort(); console.log(arr); // [1, 2, 3]
reverse	Inverse l'ordre des éléments d'un tableau.	`let arr = [1, 2, 3]; arr.reverse(); console.log(arr); // [3, 2, 1]
…		

- **Les Chaînes de caractères:**
  - Les chaînes de caractères sont définies en utilisant des guillemets simples (') ou des guillemets doubles (").
  - Les chaînes de caractères peuvent être concaténées avec l'opérateur +.
  - Les chaînes de caractères peuvent être déclarées avec l'aide de l'expression *template*, qui permet d'inclure des variables dans une chaîne de caractères.
  - Les chaînes de caractères disposent de nombreuses méthodes pour effectuer des opérations courantes, comme la recherche de sous-chaînes, la modification de la casse, la conversion en majuscules ou minuscules, etc.
  - Voici un tableau récapitulatif des méthodes couramment utilisées pour manipuler les chaînes de caractères (strings) en JavaScript

Méthode	Description	Exemple
charAt	Renvoie le caractère à l'index spécifié dans la chaîne.	let str = 'Hello'; console.log(str.charAt(1)); // 'e'
charCodeAt	Renvoie le code ASCII du caractère à l'index spécifié dans la chaîne.	let str = 'Hello'; console.log(str.charCodeAt(1)); // 101
concat	Concatène deux ou plusieurs chaînes de caractères et renvoie une nouvelle chaîne.	let str1 = 'Hello'; let str2 = 'world'; let str3 = str1.concat(' ', str2); console.log(str3); // 'Hello world'
indexOf	Renvoie l'index de la première occurrence de la sous-chaîne spécifiée dans la chaîne, ou -1 si la sous-chaîne n'est pas trouvée.	let str = 'Hello world'; console.log(str.indexOf('world')); // 6
lastIndexOf	Renvoie l'index de la dernière occurrence de la sous-chaîne spécifiée dans la chaîne, ou -1 si la sous-chaîne n'est pas trouvée.	let str = 'Hello world, hello world'; console.log(str.lastIndexOf('world')); // 14
slice	Extrait une portion de la chaîne et renvoie une nouvelle chaîne.	let str = 'Hello world'; console.log(str.slice(6, 11)); // 'world'
substring	Extrait une portion de la chaîne et renvoie une nouvelle chaîne.	let str = 'Hello world'; console.log(str.substring(6, 11)); // 'world'
substr	Extrait une portion de la chaîne et renvoie une nouvelle chaîne.	let str = 'Hello world'; console.log(str.substr(6, 5)); // 'world'
replace	Remplace toutes les occurrences d'une sous-chaîne dans une chaîne par une autre sous-chaîne.	let str = 'Hello world'; console.log(str.replace('world', 'universe')); // 'Hello universe'
toLowerCase	Convertit tous les caractères de la chaîne en minuscules.	let str = 'Hello World'; console.log(str.toLowerCase()); // 'hello world'
toUpperCase	Convertit tous les caractères de la chaîne en majuscules.	let str = 'Hello World'; console.log(str.toUpperCase()); // 'HELLO WORLD'
…		

- **Les Événements:**
  - En JavaScript, les événements sont des actions qui se produisent dans une page web (par exemple, le clic sur un bouton, le chargement de la page, etc.). On peut écrire le code qui sera exécuté lorsqu'un événement se produit, afin d'ajouter de la interactivité à notre site web.
  - Il existe de nombreux événements en JavaScript, le tableau suivant présente quelques exemples courants :

Événement	Description
click	Se produit lorsque l'utilisateur clique sur un élément.
submit	Se produit lorsque l'utilisateur soumet un formulaire.
focus	Se produit lorsque l'utilisateur place le curseur dans un champ de formulaire.
blur	Se produit lorsque l'utilisateur quitte un champ de formulaire.
load	Se produit lorsque la page est chargée.
mouseover	Se produit lorsque la souris entre dans l'élément.
mouseout	Se produit lorsque la souris quitte l'élément.
keydown	Se produit lorsque l'utilisateur appuie sur une touche du clavier.
keyup	Se produit lorsque l'utilisateur relâche une touche du clavier.
change	Se produit lorsque l'utilisateur modifie la valeur d'un champ de formulaire (par exemple, en sélectionnant une option dans une liste déroulante).
…	

- **Les Événements:**
  - Pour gérer les événements en JavaScript on utilise l'attribut **on...** de l'élément HTML.
  - Par exemple, pour gérer l'événement click sur un bouton, on peut écrire:

```
<button onclick="maFonction()">Cliquez ici</button>
```

  - De manière générale, la syntaxe est la suivante :

```
<mon-element onmon-evenement="maFonction()">
```

  - Voici quelques exemples d'utilisation de l'attribut **on...** :

```
<button onclick="maFonction()">Cliquez ici</button>
<form onsubmit="maFonction()">
  <!-- Contenu du formulaire -->
</form>
<input type="text" onfocus="maFonction()">
<input type="text" onblur="maFonction()">
```

- **Le DOM:**
  - Le DOM (Document Object Model) est un modèle de document qui représente la structure d'une page web en tant qu'arbre de nœuds.
  - Chaque élément HTML de la page est représenté par un nœud *(node)* du DOM, et chaque nœud a des propriétés et des méthodes qui permettent de manipuler l'élément HTML correspondant.
  - Le DOM est important car il permet de travailler avec le contenu et le style d'une page web en utilisant du code JavaScript. Cela permet de créer des sites web interactifs et de réaliser de nombreuses fonctionnalités intéressantes.

- **Le DOM:**
  - Par exemple, grâce au DOM, on peut:
    - Modifier le contenu d'un élément HTML en utilisant la propriété *innerHTML*.
    - Modifier le style d'un élément HTML en utilisant la propriété *style*.
    - Ajouter de nouveaux éléments HTML à la page en utilisant la méthode *createElement()* et la méthode *appendChild()*.
    - Supprimer des éléments HTML de la page en utilisant la méthode *removeChild()*.
    - …
  - Le DOM est un modèle de document dynamique, ce qui signifie qu'il peut être modifié en temps réel en utilisant le code JavaScript. Cependant, il est important de manipuler le DOM de manière efficace afin de ne pas ralentir inutilement le site web.

- **Accéder aux éléments du DOM:**
  - Il existe plusieurs façons d'accéder aux éléments du DOM en JavaScript :
  - La méthode *getElementById()* permet d'accéder à un élément en fonction de son attribut *id*.
  - Exemple :

```
<div id="mon-id">Contenu</div>
```

```
let monElement = document.getElementById('mon-id');
console.log(monElement); // <div id="mon-id">Contenu</div>
```

- **Accéder aux éléments du DOM:**
  - La méthode *getElementsByTagName()* permet d'accéder à tous les éléments d'un certain type de balise (par exemple, tous les div).
  - Exemple :

```
<div>Contenu 1</div>
<div>Contenu 2</div>
<div>Contenu 3</div>
```

```
let mesElements = document.getElementsByTagName('div');
console.log(mesElements); // [<div>Contenu 1</div>, <div>Contenu 2</div>,
<div>Contenu 3</div>]
```

- **Accéder aux éléments du DOM:**
  - La méthode *getElementsByClassName()* permet d'accéder à tous les éléments qui ont une certaine classe (par exemple, tous les éléments qui ont la classe *ma-classe*).
  - Exemple :

```
<div class="ma-classe">Contenu 1</div>
<div class="ma-classe">Contenu 2</div>
<div class="ma-classe">Contenu 3</div>
```

```
let mesElements = document.getElementsByClassName('ma-classe');
console.log(mesElements); // [<div class="ma-classe">Contenu 1</div>, <div
class="ma-classe">Contenu 2</div>, <div class="ma-classe">Contenu 3</div>]
```

  - Il est important de noter que les méthodes *getElementsByTagName()* et *getElementsByClassName()* retournent une liste d'éléments (également appelée "*HTMLCollection*"), même si cette liste ne contient qu'un seul élément.

- **Modifier les éléments du DOM:**
  - Une fois que nous avons accédé à un élément du DOM en JavaScript, on peut le modifier en utilisant ses propriétés et ses méthodes.
  - Pour changer le contenu de l'élément, on peut utiliser la propriété *innerHTML* :

```html
<div id="mon-id">Contenu initial</div>
```

```javascript
let monElement = document.getElementById('mon-id');
monElement.innerHTML = "Nouveau contenu";
console.log(monElement.innerHTML); // "Nouveau contenu"
```

- **Modifier les éléments du DOM:**
  - Pour changer le style de l'élément, on peut utiliser la propriété *style* :

```
<div id="mon-id">Contenu</div>
```

```
let monElement = document.getElementById('mon-id');
monElement.style.color = "red";
monElement.style.fontSize = "20px";
```

  - Il est important de noter que la propriété *style* permet de modifier le style de l'élément en utilisant des propriétés CSS.
  - On peut donc utiliser n'importe quelle propriété CSS valide (par exemple, *color*, *font-size*, *background-color*, etc.). On peut également utiliser la syntaxe *camelCase* (par exemple, *fontSize* au lieu de *font-size*) pour accéder aux propriétés qui comprennent un tiret (-).

- **Créer de nouveaux éléments du DOM:**
  - Il est possible de créer de nouveaux éléments du DOM en JavaScript en utilisant la méthode *createElement()*.
  - Cette méthode prend en argument le nom de la balise HTML que vous souhaitez créer (par exemple, '*div*', '*p*', '*h1*', etc.) et retourne un nouvel élément du DOM correspondant :

```
let monNouvelElement = document.createElement('div');
console.log(monNouvelElement); // <div></div>
```

  - On peut ajouter le nouvel élément à la page en l'insérant dans un autre élément du DOM existant en utilisant la méthode *appendChild()*.
  - Cette méthode prend en argument le nouvel élément à ajouter et l'ajoute en tant qu'enfant de l'élément existant :

- **Créer de nouveaux éléments du DOM:**

```
<div id="mon-id-parent">
  <div>Contenu 1</div>
  <div>Contenu 2</div>
</div>
```

```
let monElementParent = document.getElementById('mon-id-parent');
let monNouvelElement = document.createElement('div');
monElementParent.appendChild(monNouvelElement);
console.log(monElementParent); // <div id="mon-id
```

- **Supprimer des éléments du DOM:**
  - Il est possible de supprimer des éléments du DOM en JavaScript en utilisant la méthode *removeChild()*.
  - Cette méthode prend en argument l'élément à supprimer et le retire de la page.
  - Pour utiliser cette méthode, on doit récupérer l'élément parent de l'élément à supprimer, puis appeler *removeChild()* sur cet élément parent en lui passant l'élément à supprimer en argument:

```
<div id="mon-id-parent">
  <div>Contenu 1</div>
  <div id="mon-id-enfant">Contenu à supprimer</div>
  <div>Contenu 2</div>
</div>
```

- **Supprimer des éléments du DOM:**

```
let monElementParent = document.getElementById('mon-id-parent');
let monElementEnfant = document.getElementById('mon-id-enfant');
monElementParent.removeChild(monElementEnfant);
console.log(monElementParent); // <div id="mon-id-parent">
                                  //   <div>Contenu 1</div>
                                  //   <div>Contenu 2</div>
                                  // </div>
```

- Il est important de noter que la méthode *removeChild()* ne fonctionnera que si l'élément à supprimer est bien un enfant de l'élément parent.
- Si l'élément à supprimer n'a pas d'élément parent, cette méthode ne fera rien.